MAE 263F – Homework 3

Yuchen Wang (Ph.D. in Mechanical Engineering)
University of California, Los Angeles
November 2, 2025

I. Method & Algorithms

Question: A clear step-by-step description of your method, including pseudocode/algorithms for: (i) force/energy evaluation, (ii) time stepping, (iii) enforcement of Dirichlet constraints, and (iv) the path-planning or control law mapping the tracking error of node j to $\{x_c, y_c, \theta_c\}$.

Answer:

(i) Force/Energy Evaluation

Geometry: Let N be odd, $\Delta L = L/(N-1)$, $q = [x_1, y_1, \ldots, x_N, y_N]^T$, $u = \dot{q}$. For each segment:

$$r_k = \begin{bmatrix} x_{k+1} - x_k \\ y_{k+1} - y_k \end{bmatrix}, \quad \ell_k = \|r_k\|.$$

Stretching:

$$\mathscr{E}_s = \sum_{k=1}^{N-1} \frac{EA}{2\Delta L}(\ell_k - \Delta L)^2, \quad F_s = -\nabla_q \mathscr{E}_s.$$

Bending: Define turning angle $\theta_k = \text{atan2}(\det[r_{k-1}, r_k], r_{k-1} \cdot r_k)$.

$$\mathscr{E}_b = \sum_{k=2}^{N-1} \frac{EI}{2\Delta L}\,\theta_k^2, \quad F_b = -\nabla_q \mathscr{E}_b.$$

Total internal force $F_{\text{int}} = F_s + F_b$, stiffness $K = K_s + K_b$.

External: Gravity $W = [0, -mg]^T$; optional viscous damping $F_d = C\,u$.

(ii) Time Stepping

Implicit backward-Euler/Newmark-β ($\beta = \frac{1}{4}$, $\gamma = \frac{1}{2}$):

$$u^{n+1} = \frac{q^{n+1} - q^n}{\Delta t},$$

$$R(q^{n+1}) = \frac{M}{\Delta t}\left(\frac{q^{n+1} - q^n}{\Delta t} - u^n\right) + F_{\text{int}}(q^{n+1}) + C\,u^{n+1} - W = 0.$$

Newton iteration:

$$J = \frac{M}{\Delta t^2} + K(q^{n+1}) + \frac{C}{\Delta t}, \quad q^{n+1} \leftarrow q^{n+1} - J^{-1}R.$$

Pseudocode:

```
for each step:
  assemble F_int, K, W, C
  form residual R(q)
  while not converged:
      J = M/dt^2 + K + C/dt
      apply Dirichlet mask
      q = -solve(J_ff, R_f)
      q_f += q; overwrite fixed DOFs
  u = (q - q_prev)/dt
```

(iii) Dirichlet Constraints

Implemented mode (notebook): Clamp left end and prescribe the middle node to its target:

$$x_1 = y_1 = 0, \qquad x_* = x_*^\star(t),\ y_* = y_*^\star(t), \quad * = \tfrac{N+1}{2}.$$

We partition $q = [q_f; q_d]$. Each iteration overwrites $q_d \leftarrow q_d^{\text{prescribed}}$ and solves for free DOFs only.

Alternative (robot mode):

$$x_N = x_c,\ y_N = y_c, \quad x_{N-1} = x_c - \Delta L\cos\theta_c,\ y_{N-1} = y_c - \Delta L\sin\theta_c.$$

Masking pseudocode:

```
fixed = [x1, y1, x*, y*]
free  = all - fixed
q[fixed] = prescribed
R, J = restrict(R,J,free)
q = -J_ff^{-1} R_f
```

(iv) Path-Planning / Control Law

In the present implementation, I impose the middle node as a Dirichlet constraint at each step (hard tracking) and do not employ a path–planning/control law that maps the node-j tracking error to $\{x_c, y_c, \theta_c\}$.

II. Control Inputs Over Time

Question: Plots of the control inputs $x_c(t), y_c(t), \theta_c(t)$ over time.

Answer:

I changed the time horizon from $t \in [0, 1000]$ to $t \in [0, 1]$ to allow a smaller Δt while keeping the same trajectory. The target becomes

$$x_*(t) = \frac{L}{2}\cos\left(\frac{\pi}{2}t\right), \qquad y_*(t) = -\frac{L}{2}\sin\left(\frac{\pi}{2}t\right), \qquad t \in [0, 1],$$

which is equivalent to the original definition with $t \mapsto t/1000$, so the results remain the same.

Here are plots of the control inputs $x_c(t), y_c(t), \theta_c(t)$ over time.

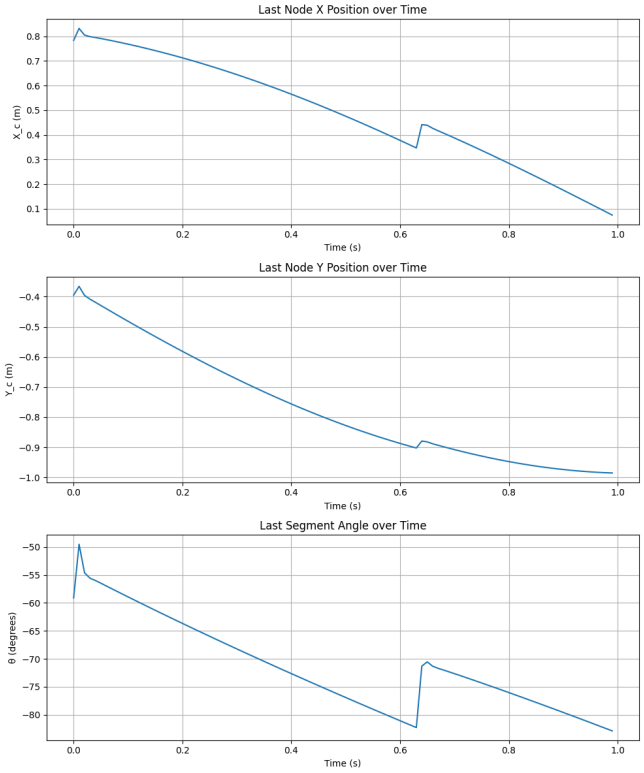

Fig. 1. plots of the control inputs $x_c(t), y_c(t), \theta_c(t)$ over time

III. Beam Shape Snapshots

Question: At least five snapshots of the beam shape (node positions) during the motion.

Answer:

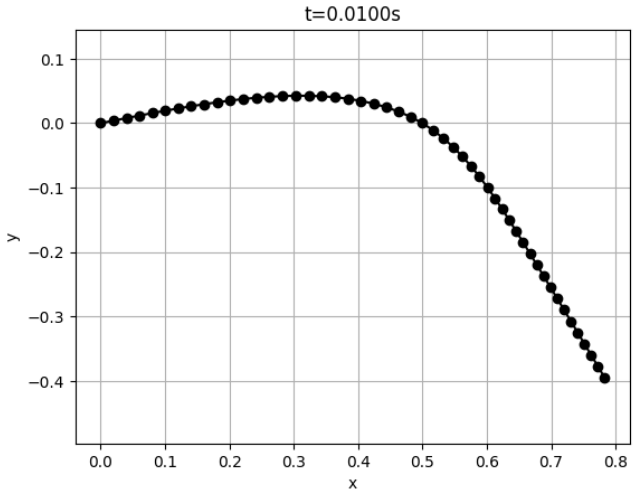

Fig. 2. plot of beam shape at $t = 0.01$ s

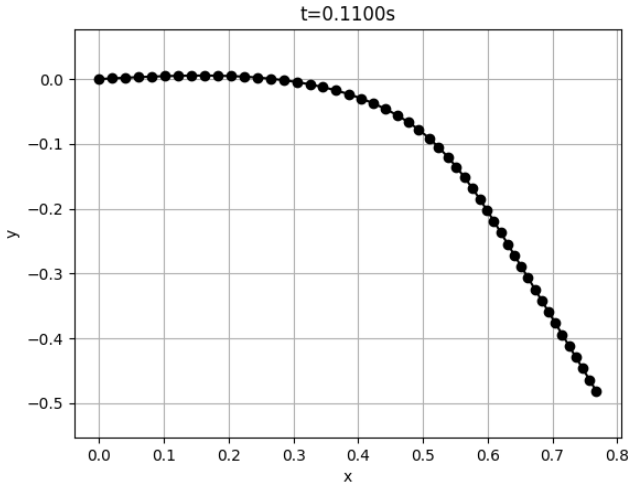

Fig. 3. plot of beam shape at $t = 0.11$ s

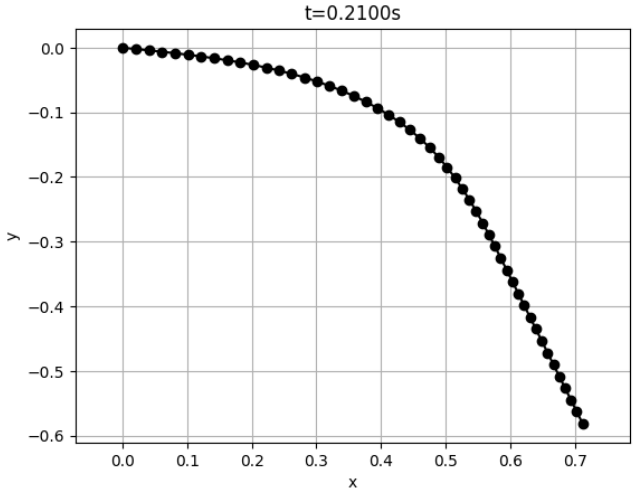

Fig. 4. plot of beam shape at $t = 0.21$ s

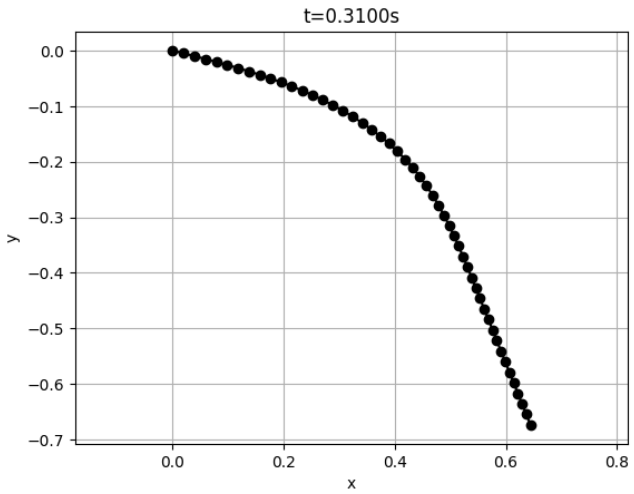

Fig. 5. plot of beam shape at $t = 0.31$ s

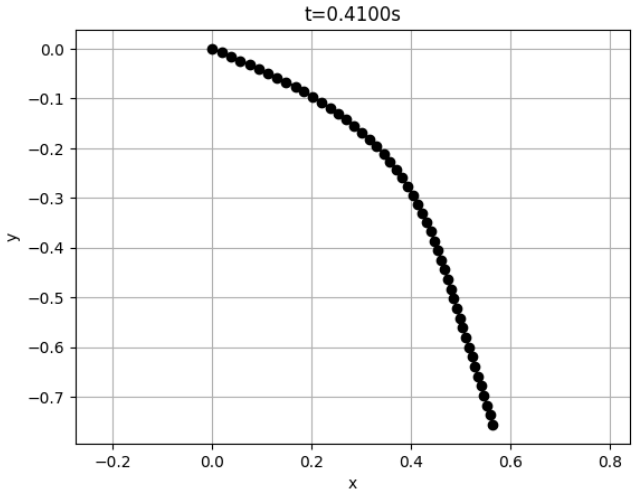

Fig. 6. plot of beam shape at $t = 0.41$ s

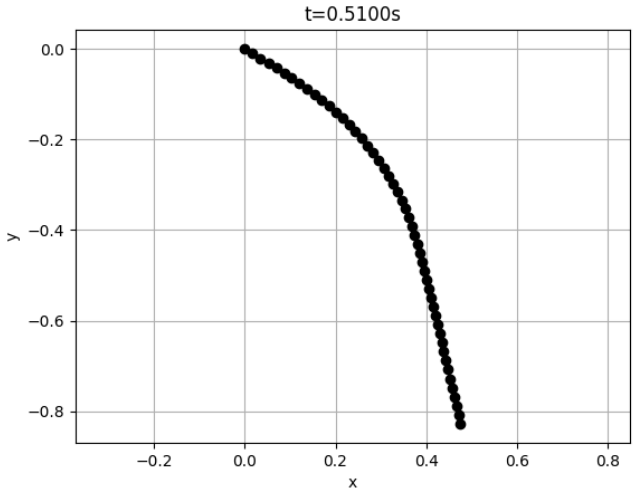

Fig. 7. plot of beam shape at $t = 0.51$ s

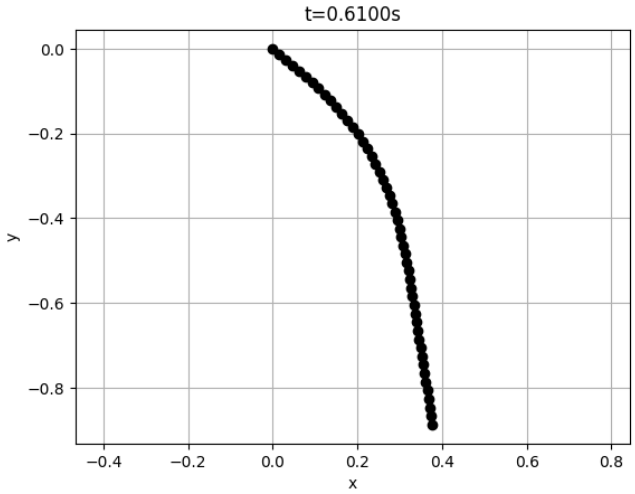

Fig. 8. plot of beam shape at $t = 0.61$ s

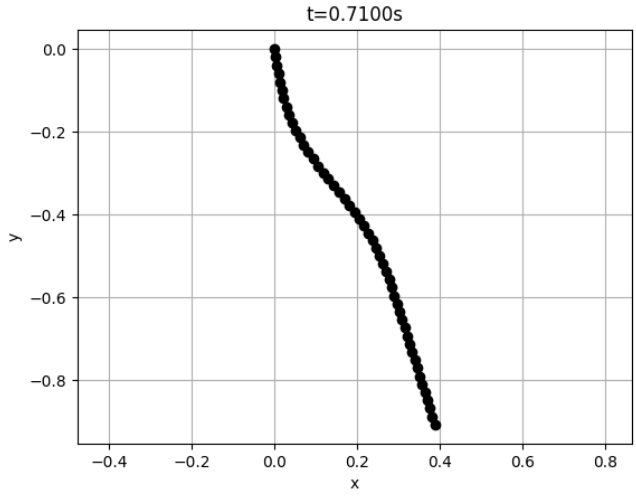

Fig. 9. plot of beam shape at $t = 0.71$ s

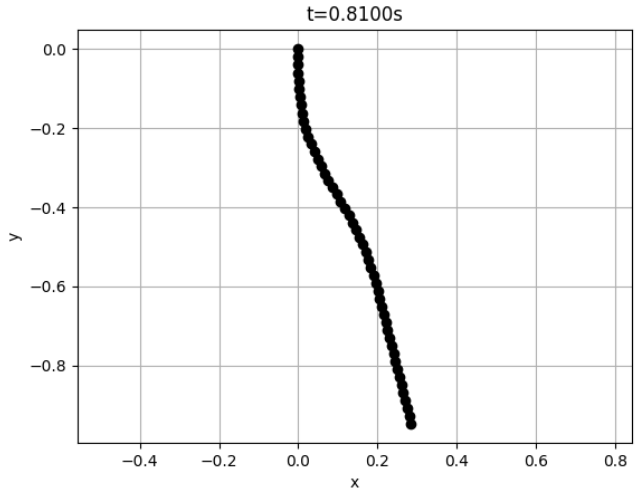

Fig. 10. plot of beam shape at $t = 0.81$ s

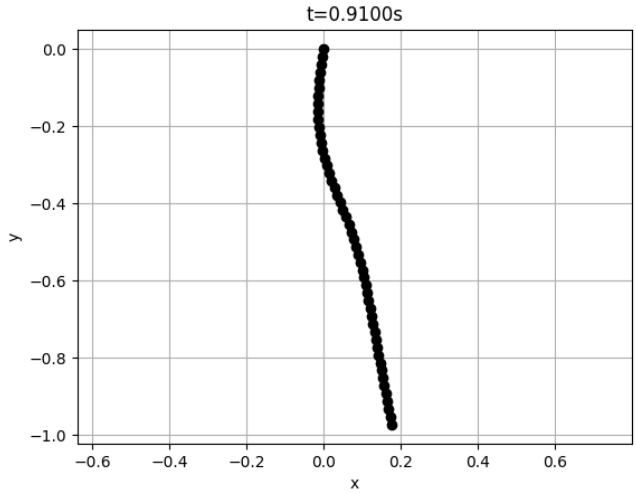

Fig. 11. plot of beam shape at $t = 0.91$ s

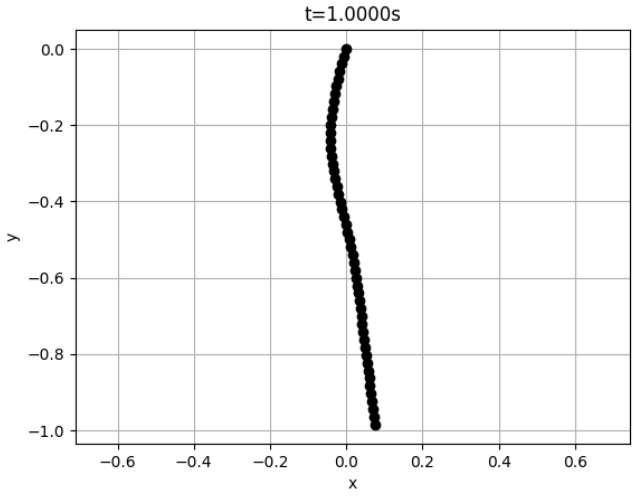

Fig. 12. plot of beam shape at $t = 1.00$ s

IV. Feasibility & Limits

Question: A brief discussion of feasibility limits due to the robot's workspace and joint limits; explain how you handle infeasible commands (e.g., saturation or trajectory re-timing).

Answer: In practice, the end-effector (right end of the beam) is subject to geometric and mechanical limits that constrain the admissible commands $\{x_c, y_c, \theta_c\}$.

a) Workspace limits: The reachable region of the robot defines a bounded workspace

$$(x_c, y_c) \in \mathscr{W} = \{(x,y) \,|\, x_{\min} \le x \le x_{\max},\ y_{\min} \le y \le y_{\max}\}.$$

If the desired command lies outside $\mathscr{W}$, it is projected back to the nearest feasible point on the boundary:

$$(x_c, y_c) \leftarrow \mathrm{Proj}_{\mathscr{W}}(x_c, y_c).$$

b) Joint and angular limits: The rotation of the end-effector is bounded by

$$\theta_{\min} \le \theta_c \le \theta_{\max},$$

and its rate of change is limited to

$$|\dot{\theta}_c| \le \omega_{\max}.$$

Violating these limits could correspond to physical self-collision or actuator saturation.

c) Saturation handling: We apply rate limiting to each component of the control increment $\Delta c = [\Delta x_c, \Delta y_c, \Delta\theta_c]^T$:

$$\Delta c \leftarrow \mathrm{clip}\big(\Delta c;\ |\Delta c_i| \le \Delta c_{\max}\big),$$

so the robot moves only within feasible velocity bounds even when the control demand is large.

d) Trajectory re-timing: If the planned trajectory requires motion beyond rate limits, the timing is uniformly scaled to maintain the same geometric path while reducing speed:

$$t' = \alpha t, \qquad \alpha = \frac{\|\Delta c\|_{\text{actual}}}{\|\Delta c\|_{\text{demand}}} < 1.$$

This ensures smooth, feasible motion without overshoot.

By combining geometric projection, saturation, and trajectory re-timing, the controller guarantees that the commanded $\{x_c, y_c, \theta_c\}$ remain inside the physical workspace and within joint constraints, while preserving stability and continuity of motion.

Code and Source Attribution

The source code is adapted from *Simulation of a Beam with Fixed DOFs* and has been refined and formatted with the assistance of ChatGPT and Gemini.